

cornea__simple__mech__model

Hyperelastic finite-element simulations built on FEniCSx (dolfinx 0.11), for two kinds of mesh:

- **2D axisymmetric:** a meridian section (r, z) of the cornea under intraocular pressure (IOP),
- **3D:** any gmsh volume mesh (tetrahedra or hexahedra), with a unit-cube test case.

The `Simulation_Framework` package is a thin layer over dolfinx. It handles:

- mesh reading (physical tags and an optional per-cell local reference system),
- material law and weak form,
- boundary conditions and the Newton solver,
- outputs (XDMF fields, CSV scalars, metadata) and plots.

A simulation is a short script: a mesh, a JSON parameter file, a list of boundary conditions and the quantities you want to record. The same script structure works in 2D axisymmetric and in 3D; only the framework class changes.

Author: B. Sauty

1. Installation (Docker)

FEniCSx is run inside a Docker container built from the provided `Dockerfile`. The image and the container are both named `fenicsx_cornea`.

Prerequisites

- Docker (Docker Desktop on Windows / macOS)

Build the image

From the root of the repository:

```
docker build -t fenicsx_cornea .
```

This starts from `dolfinx/dolfinx:v0.11.0` (dolfinx, PETSc, MPI, gmsh) and adds a LaTeX installation (matplotlib text rendering), OpenGL / Xvfb libraries (pyvista off-screen rendering) and the Python packages pandas, scipy, matplotlib, h5py, imageio, pyvista, jupyter and spyder-kernels.

Create and start the container

The repository is mounted as `/workspace`, so files are shared both ways: results written by the container appear in your local folder.

Linux / macOS:

```
docker run -it --name fenicsx_cornea -p 8888:8888 \
-v "$(pwd)"/workspace fenicsx_cornea
```

Windows (PowerShell):

```
docker run -it --name fenicsx_cornea -p 8888:8888 `
-v "${PWD}:/workspace" fenicsx_cornea
```

Come back to the same container later with `docker start -ai fenicsx_cornea`.
Check the installation:

```
python3 -c "import dolfinx; print(dolfinx.__version__)"
```

Jupyter (optional)

```
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

Then open the `http://127.0.0.1:8888/?token=...` link printed in the terminal.

Post-processing without Docker

PlotManager and the results reader do **not** need dolfinx or MPI. On a machine without Docker, a plain Python environment is enough:

```
pip install numpy matplotlib h5py pillow
```

2. Repository structure

```
cornea_simple_mech_model/
|-- Dockerfile
|-- README.md
|-- Simulation_Framework/
|   |-- __init__.py           # the library
|   |-- hyperelastic_framework.py # lazy exports
|   |-- mesh_io.py           # the two framework classes
|   |-- parameters_class.py   # .msh reader: tags + local reference system
|   |-- MechParams: material parameters and psi
|-- Postprocessing_managers/
|   |-- outputs_manager.py     # OutputManager (needs dolfinx)
|   |-- output_utils.py        # point_probe, average (need dolfinx)
|   |-- results_reader.py      # reads XDMF/HDF5/CSV/JSON (no dolfinx)
|   |-- plots_manager.py       # PlotManager, compare_scalars (no dolfinx)
|   |-- plots_manager_old.py   # previous version, not exported
|-- Usecases/
|   |-- main_cornea_axi.py     # the cornea (axisymmetric)
|   |-- plot_cornea.py         # runs the IOP simulation
|   |-- cornea.msh             # figures from the results
|   |-- mech_params_Giammarini2026.json # 2D (r, z) mesh, gmsh 2.2
```

```

|   `-- mech_params_Pandolfi2006.json
|-- Testing_tools/
|   |-- make_test_meshes.py
|   |-- main_cubic_test.py
|   |-- plot_cubic.py
|   |-- cube_hexa.msh, cube_tetra.msh
|   `-- mech_params_cube.json
`-- results/
    |-- Usecases/
    `-- Testing_tools/

```

the 3D cube test
writes cube_hexa.msh / cube_tetra.msh
runs the cube under load
created by the runs, mirrors the folders above

Public API

Everything is importable from the package root:

```

from Simulation_Framework import (Hyperelastic_axisymmetric_framework,
                                   Hyperelastic_3D_framework,
                                   read_mesh, MeshData, MechParams,
                                   OutputManager, point_probe, average,
                                   PlotManager, compare_scalars)

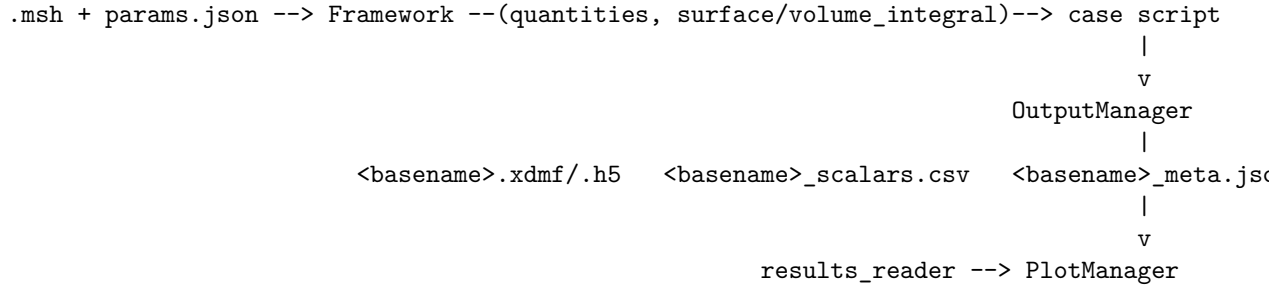
```

Name	Module	Role
Hyperelastic_axisymmetric_framework	hyperelastic_framework	2D (r, z) mesh, axisymmetric kinematics in (r, z, θ)
Hyperelastic_3D_framework	hyperelastic_framework	3D mesh, Cartesian (x, y, z)
read_mesh, MeshData	mesh_io	Reads a .msh: dolfinx mesh, facet and cell tags, local reference system
MechParams	parameters_class	Material parameters (dataclass, validated) and the strain energy density
OutputManager	outputs_manager	Writes fields (XDMF) and scalars (CSV) at each step, plus a metadata file
point_probe, average	output_utils	Helpers to build case-specific scalar outputs
PlotManager, compare_scalars	plots_manager	Reads the result files and plots them

Imports are **lazy**: a module is loaded only when one of its names is used, so

```
from Simulation_Framework import PlotManager works without dolfinx.
```

How the pieces fit



The framework knows the physics, `OutputManager` knows nothing about it, and `PlotManager` only reads files. The sidecar `<basename>_meta.json` is what lets the plots know the coordinate labels and whether the run is axisymmetric.

3. Usage

All commands are run **from the repository root** (the scripts use paths such as `Usecases/cornea.mesh` and write to `results/...`). Simulations run inside the container.

Axisymmetric cornea

```
python3 Usecases/main_cornea_axi.py
mpirun -n 4 python3 Usecases/main_cornea_axi.py    # in parallel
python3 Usecases/plot_cornea.py                    # figures, no dolfinx needed
```

The script ramps the IOP from 0 to 20 mmHg in 20 load steps (step 0 is the reference state) and writes, in `results/Usecases/`:

- `cornea_IOP_Giammarini.xdmf / .h5`: fields,
- `cornea_IOP_Giammarini_scalars.csv`: scalars,
- `cornea_IOP_Giammarini_meta.json`: metadata,
- a quick plot of the apex displacement.

3D cube test

```
python3 Testing_tools/make_test_meshes.py Testing_tools    # only to regenerate the meshes
python3 Testing_tools/main_cubic_test.py
python3 Testing_tools/plot_cubic.py
```

The unit cube is clamped on three faces ($u_x = 0$ on x_0 , $u_y = 0$ on y_0 , $u_z = 0$ on z_0) and loaded by a follower traction on z_1 , ramped to 500

mmHg. It writes `results/Testing_tools/cubetest*`. Switch `cube_hexa.msh` to `cube_tetra.msh` in the script to compare cell types.

Units are consistent everywhere: lengths in mm, stresses in MPa (1 mmHg = 1.33322e-4 MPa).

Axisymmetric vs 3D at a glance

	Hyperelastic_axisymmetric_3D_framework	Hyperelastic_3D_framework
Mesh	2D, $r \geq 0$	3D
Displacement space	P2 vector, 2 components	P2 vector, 3 components
Gradient	∇u plus hoop term u_r/r	∇u
Integrals	weighted by r , volume_integral / surface_integral include the factor 2π (full revolved body)	plain integrals
Output axes	(r, z, t) , $\mathbf{t}$ = hoop	(x, y, z)
Dirichlet axis	$0 = r, 1 = z$	$0 = x, 1 = y, 2 = z$
Tensor outputs	3×3 in (r, z, θ)	3×3 in (x, y, z)
Extra quantity	hoop_stretch $= 1 + u_r/r$	none
Field plots	yes	no (ParaView)

4. Mesh

`read_mesh(mesh_file, comm)` opens a gmsh file (format 2.2) once and returns a `MeshData` with `domain`, `facet_tag`, `cell_tags` and the optional local reference system `e_1`, `e_2`, `e_3`. The geometric dimension (2 or 3) is taken from the file. The frameworks call it for you.

Physical tags

Cornea (`Usecases/cornea.msh`, 2D):

Tag	Name	Default boundary condition in <code>main_cornea_axi.py</code>
1	<code>anterior</code>	free (zero traction)
2	<code>posterior</code>	follower pressure (IOP)
3	<code>limbus</code>	clamped in r and z
4	<code>central_line</code>	symmetry axis: $u_r = 0$
10	<code>cornea</code>	the domain

Cube (`Testing_tools/cube_*.msh`, 3D):

Tag	Face	Tag	Face
1	x0	4	y1
2	x1	5	z0
3	y0	6	z1
10	the volume cube		

Local reference system (LRS)

A mesh may carry three per-cell orthonormal vector fields as gmsh `$ElementData` views, stored by the reader as DG0 functions with 3 components:

View name (either spelling)	2D (axisymmetric)	3D
<code>e_1 / e1</code>	read from the file	read from the file
<code>e_2 / e2</code>	read from the file	read from the file
<code>e_3 / e3</code>	(0, 0, 1), the hoop direction	read from the file if present, else $e_1 \times e_2$

The LRS is optional. A mesh without `e_1` and `e_2` is valid (`MeshData.has_lrs` is `False`), and isotropic laws do not need it. Fibre laws (HGO) raise an error without it. Views may cover all elements or only the cells; boundary elements are skipped. `Testing_tools/make_test_meshes.py` shows how to append the `$ElementData` blocks to a `.msh`.

Parallel behaviour of the LRS (ghost cells) has not been verified; it is correct on a single rank.

5. Material parameters

Parameters are read from a JSON file by `MechParams.from_json`:

```
{
  "sedf_type": "Mooney-Rivlin",
  "C_10": 0.5,
  "C_01": -0.175,
  "D": 0.4
}
```

sedf_type	Strain energy density ψ	Parameters
Neo-Hookean	$C_{10}(\bar{I}_1 - 3) + \frac{(J - 1)^2}{D}$	C_10, D

sedf_type	Strain energy density ψ	Parameters
Mooney-Rivlin	$C_{10}(\bar{I}_1 - 3) + C_{01}(\bar{I}_2 - 3) + \frac{(J-1)^2}{D}$	C_10, C_01, D
HGO	$C_{10}(\bar{I}_1 - 3) + C_{01}(\bar{I}_2 - 3) + \frac{1}{2D}(J^2 - 1 - 2 \ln J) + \frac{k_1}{2k_2} \sum_{i \in \{4,6\}} \mathcal{H}(E_i) [e^{k_2 E_i^2} - 1]$	C_10, C_01, D, k1, k2, kappa, a4, a6

with

- $\bar{\mathbf{C}} = J^{-2/3} \mathbf{F}^T \mathbf{F}$, $\bar{I}_1 = \text{tr } \bar{\mathbf{C}}$, $\bar{I}_2 = \frac{1}{2} [(\text{tr } \bar{\mathbf{C}})^2 - \text{tr}(\bar{\mathbf{C}}^2)]$,
- $E_i = \kappa(\bar{I}_1 - 3) + (1 - 3\kappa)(I_i - 1)$ and $I_i = \mathbf{a}_i \cdot \mathbf{C} \mathbf{a}_i$ (generalised structure tensor with dispersion κ),
- $\mathcal{H}(E_i)$ the Heaviside function: fibres only carry load in extension ($E_i > 0$).

D controls compressibility: the initial bulk modulus is $2/D$ and $D \rightarrow 0$ is the incompressible limit. The initial shear modulus is $\mu = 2(C_{10} + C_{01})$.

Fibres. a4 and a6 are 3-component vectors given **in the mesh's local reference system**, e.g. $[1, 0, 0]$ is along e_1 . They are pushed to physical space cell by cell and normalised. Parameters not used by the chosen law are ignored.

Validation (`MechParams.__post_init__`): `sedf_type` must be one of the three laws, the required parameters must be present, $D > 0$, $k_1 \geq 0$, $k_2 > 0$, $\kappa \geq 0$, and a4, a6 must be non-zero 3-vectors.

Two parameter sets are provided in `Usecases/`: `mech_params_Pandolfi2006.json` and `mech_params_Giammarini2026.json`.

6. Writing a simulation

Both frameworks have the same interface. The 3D one is shown; for the axisymmetric cornea use `Hyperelastic_axisymmetric_framework` and 2D tags (see `Usecases/main_cornea_axi.py`).

```
import numpy as np
from mpi4py import MPI
from dolfinx import fem
from petsc4py.PETSc import ScalarType

from Simulation_Framework import Hyperelastic_3D_framework, OutputManager

# 1. framework: reads the mesh once, builds the weak form
mech = Hyperelastic_3D_framework("Testing_tools/cube_hexa.msh",
```

```

"Testing_tools/mech_params_cube.json")

# 2. boundary conditions, then the solver (this order matters)
p = fem.Constant(mech.domain, ScalarType(0.0))
mech.build_BC([
    ["Dirichlet", 1, ("clamped", 0)],           # face x0: u_x = 0
    ["Dirichlet", 3, ("clamped", 1)],           # face y0: u_y = 0
    ["Dirichlet", 5, ("clamped", 2)],           # face z0: u_z = 0
    ["Neumann_follower", 6, [0, 0, p]],         # face z1: follower traction
])
mech.build_solver()

# 3. outputs: the case decides what is recorded
q = mech.quantities                               # standard UFL quantities
fields = {"displacement": q["displacement"], "cauchy_stress": q["cauchy_stress"]}
scalars = {"volume": mech.volume_integral(q["J"])}

# 4. load stepping
with OutputManager("results/my_run", mech.domain, fields, scalars,
    meshtags={"facet_tag": mech.facet_tag},
    meta=mech.output_meta) as out:
    out.write(0.0)                                # reference state
    for n, load in enumerate(np.linspace(0, 1e-2, 11)[1:], start=1):
        p.value = load
        mech.solve_one_step(n, load)
        out.write(load)                            # the "time" of the outputs is the load
                                                    # scalars as numpy arrays
history = out.get_history()

```

The order is: framework, build_BC, build_solver, **then** the outputs.
surface_integral needs the measure mech.ds created by build_BC.

Framework attributes you will use: domain, facet_tag, cell_tags,
u, mech_params, quantities, output_meta, volume_integral(expr),
surface_integral(expr, tag).

Boundary conditions

build_BC(boundary_conditions, slip_penalty=1e3) takes a list of [type,
tag, values]. The facet tags come from the mesh, mech.facet_tag.

Type	values	Meaning
Dirichlet	("clamped", axis) or (Constant, axis)	Imposed displacement component

Type	values	Meaning
Pressure	Constant p	Follower pressure normal to the deformed surface, $p > 0$ pushes into the body
Slip	None or a constant vector $\mathbf{m}$	Sliding: $\mathbf{u} \cdot \mathbf{N} = 0$ (or $\mathbf{u} \cdot \mathbf{m} = 0$), by penalty $k = \text{slip_penalty} \mu/h$
Neumann	traction vector	Dead load
Neumann_follower	reference traction vector	Follower traction (pulled back with $J\mathbf{F}^{-T}$)
Robin	$(k, \mathbf{u_ref})$	Elastic support of stiffness k

In 3D, vector values are lists such as `[0, 0, p]` and may contain **Constants**. In the axisymmetric framework they are 2-component **Constant** vectors. Each **Dirichlet** entry constrains **one** component: clamp a face fully with one entry per axis.

Solver

`build_solver()` sets up a PETSc SNES Newton solver (`newtonls`, no line search, LU with MUMPS, absolute and relative tolerances 10^{-8}) with `quadrature_degree` 4 and P2 displacement. `solve_one_step(n, load)` solves one load step, prints the iteration count and residual, and raises **RuntimeError** if SNES does not converge. The previous step is the initial guess of the next one, so ramp the load in steps small enough for Newton to converge.

7. Outputs

Standard quantities

`mech.quantities` is a dict of symbolic UFL expressions (nothing is compiled until `OutputManager` gets them); `print(mech.quantities.keys())` lists them.

Name	Meaning
<code>displacement</code>	$\mathbf{u}$ (P2 function)
<code>PK1_stress</code>	first Piola-Kirchhoff stress $\partial\psi/\partial\mathbf{F}$
<code>cauchy_stress</code>	$\boldsymbol{\sigma} = J^{-1}\mathbf{P}\mathbf{F}^T$
<code>green_lagrange</code>	$\mathbf{E} = \frac{1}{2}(\mathbf{F}^T\mathbf{F} - \mathbf{I})$
<code>J</code>	volume ratio
<code>psi</code>	strain energy density

Name	Meaning
<code>von_mises</code>	$\sqrt{\frac{3}{2} \text{dev} \boldsymbol{\sigma} : \text{dev} \boldsymbol{\sigma}}$
<code>hydrostatic_pressure</code>	$-\text{tr} \boldsymbol{\sigma} / 3$
<code>hoop_stretch</code>	$1 + u_r / r$ (axisymmetric only)

OutputManager

`OutputManager(basename, domain, fields=None, scalars=None, meshtags=None, meta=None)`

It is generic: it does not know the physics, the dimension or the tags. The **caller** defines what is recorded.

- **fields:** `{name: expression}` or `{name: (expression, (family, degree))}`. The default space is ("Lagrange", 1) for a `fem.Function` (so displacement is written as a nodal P1 field) and ("DG", 0) for any other UFL expression (cellwise constant). Vector and tensor shapes are taken from the expression; spaces are shared between fields.
- **scalars:** `{name: ufl.Form}` (integral, compiled once, summed over ranks; use `mech.volume_integral` / `mech.surface_integral`) or `{name: callable}` (zero-argument function returning a float, called on all ranks).
- **meshtags:** `{name: MeshTags}` written to the XDMF file, usually `{"facet_tag": mech.facet_tag}`.
- **meta:** pass `mech.output_meta`. Written to the sidecar JSON together with `gdim` and `tdim`.

`write(t)` interpolates and writes all fields (one XDMF time step) and appends one CSV row, flushed immediately so rows survive a later failing step. `t` is the load. It must be called on all ranks. Use it as a context manager or call `close()`.

Files written: `<basename>.xdmf` (+ `.h5`), `<basename>_scalars.csv` (first column `t`), `<basename>_meta.json`. Missing parent folders are created.

Helpers for case-specific scalars

Built on `dolfinx`; both are collective (call on all ranks).

```
from Simulation_Framework import point_probe, average
```

```
# value of a function at a fixed reference point (2D or 3D): all components
u_apex = point_probe(mech.domain, mech.u, (0.0, z_apex))
scalars["apex_uz"] = lambda: u_apex()[1]
```

```
# mean of a quantity over a surface (or a volume)
one = fem.Constant(mech.domain, ScalarType(1.0))
scalars["uz_top"] = average(mech.surface_integral(q["displacement"][2], 6),
                             mech.surface_integral(one, 6), mech.domain.comm)
```

Forces, volumes and energies from the axisymmetric framework are for the full revolved body (integrated over 2π). The limbus reaction in `main_cornea_axi.py` is an example of a surface integral of $\mathbf{P} \cdot \mathbf{N}$.

Examples of case definitions: `cornea_outputs` in `Usecases/main_cornea_axi.py` (apex displacements, central thickness, limbus reaction, volume) and `cube_outputs` in `Testing_tools/main_cubic_test.py` (volume, strain energy, mean u_z and σ_{zz}).

8. Plotting

`PlotManager` reads the result files only (numpy, h5py, matplotlib). It adapts to the run from `<basename>_meta.json`: coordinate labels and component names are `r`, `z`, `t` (hoop) for an axisymmetric run and `x`, `y`, `z` otherwise.

```
import matplotlib.pyplot as plt
from Simulation_Framework import PlotManager, compare_scalars

pm = PlotManager("results/Usecases/cornea_IOP_Giammarini",
                 labels={"apex_uz_anterior": r"anterior apex $u_z$"})
pm.info() # what is in the files

# scalar histories: vs load t, or one scalar vs another (any dimension)
pm.plot_scalars(["apex_uz_anterior", "apex_uz_posterior"])
pm.plot_scalars("limbus_reaction_z", x="apex_uz_anterior")

# maps on the (r, z) section (2D only)
pm.plot_field("von_mises", deformed=True, mirror=True)
pm.plot_field("cauchy_stress", component="tt", symmetric=True) # hoop stress

# profiles along the symmetry axis: last step, then all steps
pm.plot_profile("green_lagrange", component="zz")
pm.plot_profile("green_lagrange", component="zz", t="all")

pm.animate_field("von_mises", "vm.gif", deformed=True, mirror=True)

# parameter study
runs = [PlotManager(f"results/run_{k}", label=f"run {k}") for k in range(3)]
compare_scalars(runs, "apex_uz_anterior")
plt.show()
```

Method	Purpose	Dimension
<code>info()</code>	Prints dimension, axes, scalars, fields and their time range	any
<code>plot_scalars(y, x="t")</code>	One or several scalars against t or another scalar	any
<code>plot_field(name, ...)</code>	Colour map at one load step	2D
<code>sample(name, points, ...)</code>	Field values at reference points	2D
<code>plot_profile(name, ...)</code>	Field along a line (default: the symmetry axis)	2D
<code>animate_field(name, filename, ...)</code>	<code>.gif</code> (Pillow) or <code>.mp4</code> (ffmpeg) over all steps	2D
<code>compare_scalars(manager, y)</code>	Overlay runs	any
<code>quick_look(basename)</code>	One panel per scalar (call <code>plt.show()</code> after)	any

Common arguments

- `t`: the load step (default: the last one; the nearest stored step is used, with a warning if it differs).
- `component`: vectors take an axis label ('r', 'z', 'x'...) or 'magnitude'; tensors take two labels ('rr', 'zz', 'tt', 'rz', 'xy'...).
- `deformed=True`, `scale=...`: draw on the deformed configuration (needs the displacement field).
- `mirror=True`: also draw the $r < 0$ half (axisymmetric runs only).
- `symmetric=True`: colour range centred on 0 with a diverging colormap.
- `ax=...`: draw into your own axis (every method returns its axis); `save="file.png"`: create, save and close a stand-alone figure. Do not combine the two. Nothing calls `plt.show()` for you.
- Profiles in a non-axisymmetric run need explicit `start` and `end` points.

Label names for case-specific scalars go in `PlotManager(labels={...})`; the standard quantities are labelled by the library (`DEFAULT_LABELS`).

3D results

Field maps, profiles and animations are 2D only: `PlotManager` raises `NotImplementedError` on a 3D run. Scalar histories work in 3D, and the fields are meant to be opened in ParaView from `<basename>.xdmf` (the facet tags are written there too). `results_reader` already reads 3D meshes (tetrahedra, hexahedra), so 3D plotting can be added without touching the file format.

Reading results directly

`results_reader` (numpy and h5py only) exposes `read_scalars`, `read_meta`, `XDMFResults` (mesh, cells, fields, time series, `values(name, t)`) and `extract_component`, for custom post-processing.

9. Extending the library

- **New output:** no registry to edit. Add an entry to the `fields / scalars` dicts of your case, or a new key to `quantities` in the framework to share it between cases. Add a label in `DEFAULT_LABELS` (`plots_manager.py`) or in `PlotManager(labels=...)`.
 - **New material law:** add the name to `SedfType` and `REQUIRED`, and a branch in `MechParams.strain_energy_density_function` returning $\psi(\mathbf{F})$. Both frameworks use it unchanged, since $\mathbf{P} = \partial\psi/\partial\mathbf{F}$.
 - **New boundary condition:** add a branch in `build_BCs` of **both** frameworks (the axisymmetric one includes the factor r and in-plane vectors).
-

10. Known limitations

Found while documenting the code; worth fixing before relying on the features.

- **HGO parameters:** The volumetric term differs from the other two laws ($\frac{1}{2D}(J^2 - 1 - 2\ln J)$).
- **tdim vs gdim:** `read_mesh` takes the dimension from the file; a 2D mesh used with the 3D framework (or the reverse) is not rejected explicitly.
- **LRS in parallel:** not verified (see section 4).
- `plots_manager.py` has no command-line entry point, use `quick_look(basename)`.